CODE COVERAGE REPORT
Code Coverage Report — Sprint 2

Tester: Hanumantha L (PES2UG24CS179)

Application: `railway_sprint2.cpp`

Baseline: `fc2c53227724ae2fd914a0d1c4fb4815c1ee2727`

Environment: Linux x86-64, GCC/G++ C++17

Evidence: `docs/evidence/sprint2/`

Tool: GCC `gcov`

Command: `gcov -b -c railway_sprint2.cpp`

Source: `railway_sprint2.cpp` at `fc2c53227724ae2fd914a0d1c4fb4815c1ee2727`

Instrumentation build: `g++ -std=c++17 -Wall -Wextra -pedantic --coverage railway_sprint2.cpp -o railway_sprint2`

Measured application coverage

The report entry for the application source was:

```text
File 'railway_sprint2.cpp'
Lines executed:86.20% of 413
Branches executed:71.78% of 2672
Taken at least once:40.27% of 2672
Calls executed:66.74% of 1828
```

This report separates the application source from the many standard-library headers also printed by `gcov`. Percentages are from the actual final suite plus the Cash/Card/ownership supplemental scenarios, not estimates. GCC's **71.78%** is the percentage of instrumented

branch sites reached; its separate **40.27%** figure is the percentage of instrumented branch outcomes taken at least once. These measure different aspects of branch execution and should not be conflated with a single conventional branch-coverage percentage.

Evidence and interpretation

- Raw summary: `docs/evidence/sprint2/coverage_gcov.txt`
- Line-level annotated output: `docs/evidence/sprint2/railway_sprint2.cpp.gcov`
- Suite result: `docs/evidence/sprint2/suite_summary.txt`
- Supplemental result: `docs/evidence/sprint2/supplement_assertions.txt`

The 13.80% of application lines not executed and the uncovered branches require additional negative/error, storage-failure, and platform-specific tests. Coverage is a measurement of executed code, not a proof of correctness.

Current server test-suite measurement

Measured: 2026-11-10

Environment: Node.js v24.11.1

Command: `npm run test:coverage` (from `server/`)

Result: 3 tests passed; 0 failed.

File	Line coverage	Branch coverage	Function coverage	Uncovered lines
`server/src/services/railRadar.js`	81.48%	87.50%	50.00%	19-23

This is the JavaScript module coverage reported by Node's test runner for modules loaded by the current server tests. It is not whole-repository coverage: the C++ integration test launches a separately compiled, uninstrumented binary, and the client currently has no test

script. The Sprint 2 C++ figures above refer to `railway_sprint2.cpp` and its recorded test suite; they are a separate measurement, not coverage of the current server engine.

BRANCH COVERAGE REPORT
Branch Coverage Report — Sprint 2

Tester: Hanumantha L (PES2UG24CS179)

Application: `railway_sprint2.cpp`

Baseline: `fc2c53227724ae2fd914a0d1c4fb4815c1ee2727`

Environment: Linux x86-64, GCC/G++ C++17

Evidence: `docs/evidence/sprint2/`

Tool: GCC `gcov -b`

Command: `gcov -b -c railway_sprint2.cpp`

Measured application branch coverage

```text
File 'railway_sprint2.cpp'
Branches executed:71.78% of 2672
Taken at least once:40.27% of 2672
```

GCC reports **2,672 instrumented application branches**, with **71.78% of instrumented branch sites executed** and **40.27% of instrumented branches taken at least once**. These are different GCC metrics: the first indicates how many branch sites were reached during execution, while the second indicates how many individual branch outcomes were observed at least once. Therefore, 71.78% is not presented as conventional branch coverage without this qualification. The raw `gcov` output is committed as `docs/evidence/sprint2/coverage_gcov.txt`; the annotated source is `docs/evidence/sprint2/railway_sprint2.cpp.gcov`.

Tested decision paths

The suite exercised valid and invalid menu/input paths, duplicate registration, short passwords, three-failure lockout, invalid dates, no-route search, invalid PNRs, full-class alternative output, over-limit seat recovery, booking confirmation/no-confirmation, all simulated payment options, repeat cancellation, ownership rejection, admin access control, add/update/remove, inventory validation, reporting, persistence, and audit paths.

Uncovered or not safely executable

The percentage does not mean all semantic paths were validated. Remaining paths include filesystem open/write/rename failures, malformed persisted records, multi-process concurrency, Windows-specific password handling, platform-specific terminal behavior, some administrator failure branches, and unit-level module paths. These require additional harnesses or controlled fault injection and are not claimed as tested here.

Current server test-suite measurement

Measured: 2026-11-10

Environment: Node.js v24.11.1

Command: `npm run test:coverage` (from `server/`)

Result: 3 tests passed; 0 failed.

| File | Branch coverage |

|---|---:|

| `server/src/services/railRadar.js` | 87.50% |

This is Node's branch coverage for the JavaScript module loaded by the current server tests. It is not a repository-wide percentage: the C++ integration test runs an uninstrumented

binary, and the client currently has no test script. The 71.78% / 40.27% GCC figures above belong to the separate Sprint 2 measurement of `railway_sprint2.cpp`.